Oracle Reducto Extract API Integration Plan

Status date: June 27, 2026

This project is integrated with the Reducto Extract API. The active structured document path is `/extract`: the app sends a schema to Reducto, Reducto returns typed JSON with optional citations, and Oracle stores the schema, typed result, and raw Reducto response as JSON.

The older parse/RAG path remains available for chunk retrieval, table promotion, vectors, and evidence-backed Q&A. It is not the active structured-field integration path described in this plan.

1. Integration Objective

The integration proves that an Oracle-backed application can call Reducto Extract, receive schema-typed JSON, persist the result in Oracle 23ai, and retur
n
the same typed output to a browser or CLI caller with enough metadata to audit the run.

The proof path is:

```text
Browser demo or CLI
  -> /api/extract/url or reducto-oracledb ingest --mode extract
  -> ReductoDocumentParser.extract_url / extract_file
  -> client.extract.run(...)
  -> normalize_extract_response
  -> OracleDocumentRepository.store_extract_result
  -> DOCUMENTS + DOCUMENT_EXTRACTIONS
  -> response includes reducto_endpoint=/extract + request_body + extracted_json
 + Oracle IDs
```

2. Implemented Components

Component	Location	Implementation detail
Extract wrapper	`src/reducto_oracledb/reducto_client.py`	`ReductoDocumentP
arser.extract_url()` and `extract_file()` call `_extract()`, which calls `client
.extract.run(...)`.
Extract request options	`src/reducto_oracledb/reducto_client.py`	Schema is
sent as `instructions={"schema": schema}`; optional prompt is sent as `instruct
ions.system_prompt`; citations and Deep Extract are sent in `settings`.
Extract response normalization	`src/reducto_oracledb/reducto_client.py`	`n
ormalize_extract_response()` keeps `job_id`, `studio_link`, `raw_response`, `sch
ema_json`, `extracted_json`, and `citations_enabled`.
Oracle schema	`src/reducto_oracledb/oracle.py`, `sql/schema.sql`	`DOCUMENT
_EXTRACTIONS` stores `schema_json`, `extracted_json`, `raw_reducto_output`, `cit
ations_enabled`, and Reducto job metadata.
Oracle write path	`src/reducto_oracledb/oracle.py`	`store_extract_result()
` inserts a `DOCUMENTS` row and a linked `DOCUMENT_EXTRACTIONS` row.
CLI path	`src/reducto_oracledb/cli.py`	`reducto-oracledb ingest --mode ext
ract --schema-file schema.json ...` executes the Extract API flow and prints Ora
cle IDs.
Browser demo path	`demo/app.py`, `demo/static/*`	`POST /api/extract/url` c
alls the Extract API and returns proof fields plus typed JSON.
Demo request/response artifacts	`demo/extract_api_request.json`, `demo/extra
ct_api_response.example.json`	Reusable payload and example output for proving
`/api/extract/url` calls the Extract API and returns typed JSON.
Demo notebook	`notebooks/extract_demo_notebook.ipynb`	Runs Extract API, st

ores the result, and reads it back with `JSON_SERIALIZE(...)` to preserve JSON n
umeric types. |

3. Extract API Request Behavior

The backend builds this SDK request:

```python
response = client.extract.run(
    input=input_value,
    instructions={
        "schema": schema,
        "system_prompt": system_prompt,
    },
    parsing=financial_extract_parse_options(...),
    settings={
        "citations": {
            "enabled": citations_enabled,
            "numerical_confidence": numerical_confidence,
        },
        "deep_extract": deep_extract,
    },
)
```

Request behavior:

Input	Behavior
Public or presigned URL	Passed directly as `input` to `client.extract.run(..
.)`.
Local file	Uploaded with `client.upload(...)`; the upload handle is passed a
s `input`.
SEC URL blocked by Reducto fetch	The wrapper downloads the source locally wi
th `SEC_USER_AGENT`, uploads it to Reducto, then calls Extract API on the upload
.
Schema	Required for CLI extract mode through `--schema-file`; optional in th
e demo because the demo provides a default schema.
Citations	Enabled by default; disabled with `--no-citations` in CLI or `disa
ble_citations` in demo API payload.
Deep Extract	Disabled by default; enabled with `--deep-extract` in CLI or `d
eep_extract` in demo API payload.

The Extract API is the backend doing the structured field extraction. The app
does not synthesize these values from parse chunks.

4. Extract API Response Behavior

The wrapper expects a response with a top-level `result` value. That value is
stored unchanged as `NormalizedExtractResult.extracted_json`.

Typical returned structure with citations enabled:

```json
{
  "company_name": {
    "value": "Apple Inc.",
    "citations": [
      {
        "type": "Text",
        "bbox": {
          "page": 5,
```

```
          "left": 0.1045,
          "top": 0.8130,
          "width": 0.7908,
          "height": 0.0194
        },
        "content": "Apple Inc.",
        "confidence": "high"
      }
    ]
  },
  "fiscal_year": {
    "value": 2023,
    "citations": []
  }
}
```

The app returns the typed result to callers. It also returns metadata that proves
the Extract API path ran:

Demo response field	Meaning
`route`	Always `/api/extract/url` for browser URL extraction.
`backend_api`	Always `Reducto Extract API`.
`reducto_endpoint`	Always `/extract`, the Reducto backend route reached through the SDK.
`sdk_call`	Always `client.extract.run`.
`request_body`	The effective Extract SDK request captured by the wrapper, including `input`, `instructions`, `parsing`, and `settings`.
`extracted_json`	The typed JSON result returned by Reducto.
`document_id`	Oracle `DOCUMENTS` row created for the source document.
`extraction_id`	Oracle `DOCUMENT_EXTRACTIONS` row created for the extraction.
`reducto_job_id`	Reducto job ID returned by Extract API when present.
`studio_link`	Reducto Studio link returned by Extract API when present.

5. Oracle Persistence

`OracleSchemaManager.create_schema()` creates the tables required by the Extract
API integration:

```text
DOCUMENTS
DOCUMENT_EXTRACTIONS
```

`DOCUMENTS.raw_reducto_output` stores the complete Reducto response for document
traceability. `DOCUMENT_EXTRACTIONS` stores the extraction-specific payload:

Column	Stored value
`document_id`	Foreign key to `DOCUMENTS`.
`reducto_job_id`	Reducto Extract job ID.
`schema_json`	Exact schema sent to `instructions.schema`.
`extracted_json`	Typed JSON returned under Reducto response `result`.
`raw_reducto_output`	Complete Reducto Extract API response.
`citations_enabled`	`1` when citations were requested, otherwise `0`.
`studio_link`	Reducto Studio URL when returned.

The repository method:

```python
document_id, extraction_id = OracleDocumentRepository(connection).store_extract_
result(
    metadata,
    extract_result,
)
```

6. Browser Demo Contract

The demo now exposes the Extract API path as the first workflow.

Endpoint:

```text
POST /api/extract/url
```

Example request:

```json
{
  "url": "https://www.sec.gov/Archives/edgar/data/320193/000032019323000106/aapl
-20230930.htm",
  "company": "AAPL",
  "year": "2023",
  "filing_type": "10-K",
  "system_prompt": "Extract audited annual filing fields.",
  "schema": {
    "type": "object",
    "properties": {
      "company_name": { "type": "string" },
      "fiscal_year": { "type": "integer" },
      "filing_type": { "type": "string" },
      "total_net_sales_millions": { "type": "number" },
      "net_income_millions": { "type": "number" }
    },
    "required": ["company_name", "fiscal_year", "filing_type"]
  }
}
```

The browser renders:

1. Backend route: `/api/extract/url`
2. SDK call: `client.extract.run`
3. Reducto backend endpoint: `/extract`
4. Extract API request body
5. Extract API typed JSON result
6. Reducto job metadata
7. Oracle `document_id` and `extraction_id`

This makes the Extract API integration visible without inspecting server logs.

Demo artifacts:

- `demo/extract_api_request.json` can be posted directly to `/api/extract/url`.
- `demo/extract_api_response.example.json` shows the response contract, includin
g
  `reducto_endpoint`, `request_body`, `extracted_json`, Reducto job metadata,
  and Oracle IDs.

7. CLI Contract

CLI extract command:

```bash
reducto-oracledb ingest \
  --mode extract \
  --url "https://www.sec.gov/Archives/edgar/data/320193/000032019323000106/aapl-
20230930.htm" \
  --schema-file ./schemas/financial_extract_schema.json \
  --company AAPL \
  --year 2023 \
  --filing-type 10-K
```

CLI output includes:

```json
{
  "document_id": 83,
  "extraction_id": 3,
  "reducto_job_id": "854cc2a0-1721-493c-b493-2c336ca99217",
  "citations_enabled": true,
  "schema_fields": 8,
  "result_type": "dict"
}
```

8. Validation Evidence

Validation already performed in this repo:

Check	Result
`notebooks/extract_demo_notebook.ipynb` executed end to end	Reducto Extract job completed and wrote Oracle `document_id=83`, `extraction_id=3`.
Notebook readback	Uses `JSON_SERIALIZE(extracted_json RETURNING CLOB)` and `json.loads(...)`; numeric JSON values remain typed in Python.
Extract client unit test	Asserts schema is sent through `instructions.schema` and citations through `settings.citations`.
Oracle repository unit test	Asserts `DOCUMENT_EXTRACTIONS` insert includes schema JSON, extracted JSON, and citation flag.
Demo endpoint unit test	Asserts `/api/extract/url` response reports `Reducto Extract API`, `/extract`, `client.extract.run`, request body, output JSON, and Oracle IDs.
Demo request fixture	`demo/extract_api_request.json` contains a ready-to-run `/api/extract/url` payload.

Recommended validation commands:

```bash
.venv/bin/pytest tests/test_reducto_client.py tests/test_oracle_repository.py te
sts/test_demo_app.py
.venv/bin/ruff check .
.venv/bin/mypy .
```

9. Acceptance Criteria

The integration is complete when:

| Requirement | Acceptance check |

|---|---|
App uses Extract API for structured fields	Demo and CLI call `client.extract.run(...)`, not parse-derived extraction.
Reducto `/extract` route is visible	Demo response and UI show `reducto_endpoint=/extract`.
Request schema is visible	Demo response and UI show `request_body.instructions.schema`.
Response typed JSON is visible	Demo response and UI show `extracted_json`.
Oracle stores proof	`DOCUMENT_EXTRACTIONS` contains `schema_json`, `extracted_json`, and `raw_reducto_output`.
Citations are preserved	Citation wrappers remain inside `extracted_json`; no flattening step removes them.
PDF and markdown plan agree	`integration-plan.pdf` is generated from `integration-plan.md`.

10. Out-of-Scope Companion Path

The parse/RAG path is still useful, but it is not the active `/extract` integration:

```text
client.parse.run(...)
  -> normalize_parse_response
  -> DOCUMENT_CHUNKS / PARSED_TABLES / FINANCIAL_FACTS
  -> Oracle vector, hybrid search, and extractive Q&A
```

Use parse for retrieval and chunk-level evidence. Use Extract API for schema-defined JSON fields.